Database Connectivity Activity

Code Escape: Cloud Save Progress Prototype

Capstone-Based Prototype (Unity)

Group Number	IT22
Group Members	Jean Joseph C. Mallari Mark Daniel M. Haguisan Louis Angelo D. Junio Alynus Job C. Tombado
Course / Section	*[Course and section]*
Instructor	*[Instructor]*
Date prepared	11 October 2026

Contents: Parts I to VIII, screenshots, references.

Part I. Game Prototype Overview

Item	Answer
Group Number	IT22
Group Members	Jean Joseph C. Mallari; Mark Daniel M. Haguisan; Louis Angelo D. Junio; Alynus Job C. Tombado
Prototype Direction	Capstone-Based
Game Title	Code Escape
Game Type or Genre	Educational programming puzzle / story adventure (2D / 2.5D)
Target Users	Students learning beginner programming (Python, PHP, SQL)
Platform Used	Unity desktop game, Windows PC
Development Tool Used	Unity Editor 6000.5.10f1, C#; code editor: *[confirm: Visual Studio or VS Code]*
Database or Storage Used	Unity Cloud Save (player data, key-value) with Unity Authentication anonymous sign-in; local PlayerPrefs as the on-device save. *Not in the activity's example list: instructor approval to be confirmed.*

Main purpose. Code Escape teaches beginner programming through a mystery story. The player must answer programming questions to unlock doors, locks and hazards and move through six levels.

What the player does. The player chooses a language and a level on the level map, moves the character, reaches an obstacle, and answers a question (choosing or typing an answer, optionally using a hint). A correct answer earns points and unlocks the way forward; clearing a level unlocks the next.

Data saved. Current level, highest unlocked level, completed levels, total score, achievements, the time of the latest update, and a snapshot used to restore the level map.

Why this direction. The group already has the Code Escape project as its capstone. Saving and restoring learning progress is a real requirement of that project, so the database work is useful beyond this activity. The game was already playable, so this activity added the cloud storage to existing gameplay.

Part II. Data Requirement Analysis

Example values come from the group's test runs in the Unity Editor on 11 October 2026.

Data Item	Purpose	Example Value
Player Name	Not stored yet. The game signs in anonymously, so the player is identified only by the Unity Player ID. A name will be stored when the planned login system exists.	(none) Player ID is a text value assigned by Unity
Score (`total_score`)	Records points: +100 per solved challenge, -25 per hinted question, counted once each.	0 at the start of a new record
Level or Stage (`current_level`, `highest_unlocked_level`)	Shows where the player is and how far they have unlocked.	`neighborhood` after clearing `home`
Result or Remarks (`completed_levels`, `achievements`)	Records which levels are cleared and which achievements are unlocked.	`["home"]`; `["first_level"]`
Date or Time (`last_played`)	Time of the latest progress update.	`2026-10-10T20:01:39Z` (UTC)
Other Data	A restore snapshot (selected language, playthrough state, level progress per language) so the existing level map and Continue work after loading.	JSON text, about 1 KB in the test

Why the game stores this data. Players stop and return. Without stored progress they would repeat solved questions and levels, and the group could not show learning progress or build a score summary later.

How it improves the experience. On the next start the level map shows cleared and unlocked levels, Continue resumes the level in progress, and the Cloud progress panel shows the saved score and achievements.

Data that should not be stored. Passwords or sign-in secrets (the planned login will be handled by Unity Authentication, not by the game); the player's typed answers or code (answers are only compared with a stored solution and never executed); real names or other personal details that are not needed; question content (it belongs to the project, not to a player); and credentials or keys in the repository.

Part III. Database Design

Unity Cloud Save stores key-value items for each signed-in player, not SQL tables. The structure is one record type, `PlayerProgressData`, stored as JSON text under the key `code_escape_progression`, plus a second key, `code_escape_snapshot`, for restoring the game state. Cloud Save stores each player's items under that player, so one player's data cannot overwrite another's.

Field Name	Data Type	Description
`player_id`	Text	Unity Player ID of the owner (unique identifier)
`version`	Number	Record format version
`current_level`	Text	Level ID of the level reached
`highest_unlocked_level`	Text	Level ID of the furthest level unlocked
`completed_levels`	List of text	Cleared level IDs, no duplicates
`total_score`	Number	Accumulated score (not below 0)
`achievements`	List of text	Unlocked achievement IDs
`last_played`	Text (ISO date and time)	When the record was last updated (UTC)
`scored_challenges`, `penalised_hints`	List of text	Bookkeeping so each solved challenge and hint is counted once

Mapping to the suggested `player_scores` fields: player_id = `player_id`; player_name = not stored yet; score = `total_score`; level = `current_level`; remarks = `achievements` and `completed_levels`; created_at = `last_played`.

[Screenshot: Unity Dashboard, Cloud Save, Player Data, showing the keys code_escape_progression and code_escape_snapshot. The group already has this screenshot.]

Purpose of the structure. It keeps each student's progress, score and achievements so the game can restore them on the next start.

Unique identifier. `player_id`, the authenticated Unity Player ID. Cloud Save stores the items under that player.

Fields required before saving. `player_id` (set from the signed-in player), `version`, and the level and score fields. The game fills missing lists with empty lists, clamps the score at 0 and removes duplicate or unknown level IDs before saving. A load fails, without overwriting anything, if only one of the two keys exists or the data cannot be read.

Part IV. Database Connectivity Implementation

How the connection works

On start, `CloudProgressService` (`Assets/Scripts/Cloud`) initialises Unity Gaming Services, signs in anonymously with Unity Authentication, and loads the two keys from Cloud Save. If a record exists, it is restored into the game's save system; if not, a new record is created. When progress changes, the service builds the record and saves both keys in one request. No password or key is in the code: the project is linked to a Unity Dashboard project, and Cloud Save only accepts requests from a signed-in player.

```
// Load (CloudProgressService.cs)
var items = await CloudSaveService.Instance.Data.Player.LoadAsync(
    new HashSet<string> { KeyProgression, KeySnapshot },
    new PlayerData.LoadOptions(new PlayerData.DefaultReadAccessClassOptions()));

// Save (CloudProgressService.cs)
await CloudSaveService.Instance.Data.Player.SaveAsync(
    new Dictionary<string, object> { { KeyProgression, recordJson }, { KeySnapshot, snapshotJson } },
    new PlayerData.SaveOptions(new PlayerData.DefaultWriteAccessClassOptions()));
```

[Screenshot: relevant code or connection setup in the Unity or code editor]

Required evidence

[Screenshot 1: the game before saving data (for example the level map or a level before it is cleared)]

[Screenshot 2: a successful save, the "Saved to cloud" message in the top-right corner]

[Screenshot 3: the saved data in Unity Dashboard, Cloud Save, Player Data (open the value of code_escape_progression)]

[Screenshot 4: the game displaying the retrieved data, the "Cloud progress" panel and the level map after restarting]

Answers

What action triggers saving? Clearing a level (saved at once), unlocking a level, solving a challenge or changing the score (grouped together after 3 seconds), returning to the main menu, and the app being paused. If the data has not changed, nothing is sent.

How does the game retrieve stored data? At startup it calls Cloud Save `LoadAsync` for the signed-in player. Gameplay stays locked behind a "Connecting to Unity Cloud Save" screen until the load succeeds; on failure the screen shows the error and a Retry button.

Where is the retrieved data displayed? On the existing level map (cleared, available and locked levels), in the Continue panel of the main menu, and in a "Cloud progress" panel on the main menu and level map showing the Player ID, current and highest unlocked level, completed levels, total score, achievements and last update. *The group should confirm the panel's appearance in the screenshot, because it was built in code and its layout was not checked on screen when this report was prepared.*

Problem encountered. After the Cloud Save package was added, Unity showed a "Package Manager Error": `Packages/manifest.json` was not valid JSON because the line for `com.unity.services.cloudsave` appeared twice. Other early problems: looking for the saved data in the Dashboard's Game Data page (which is for server-

written items) instead of Cloud Save, Player Data; and the Editor pausing the game while the window was in the background, which delayed saves during testing.

How it was solved. The duplicate line was removed and the file was checked as valid JSON; Unity then loaded the packages. The Player Data page was used to view the saved keys. For testing, the game was kept running in the background for that Editor session.

Part V. Repository Information

Item	Answer
Repository Platform	GitHub (with GitHub Desktop)
Repository Name	IT22_DatabaseConnectivity
Repository Link	https://github.com/jjcmall/IT22_DatabaseConnectivity
Repository Visibility	Private
Instructor Access Confirmed	*[Yes or No: if Private, add the instructor as collaborator]*
Number of Meaningful Commits	*[Count from GitHub: at least 5 required, with clear messages]*

Files included. The Unity project: `Assets` (scenes, scripts, prefabs, art, question assets), `Packages/manifest.json` and `packages-lock.json`, `ProjectSettings`, `README.md`, a Unity `.gitignore` (Library, Temp, Logs and UserSettings are left out), and a `Documentation` folder. No password, key or service account file is in the project; Cloud Save uses the signed-in player's session, and the project ID in `ProjectSettings` is an identifier, not a secret.

Where the save and retrieve functions are. `Assets/Scripts/Cloud/CloudProgressService.cs` (initialise, sign in, `LoadAsync`, `SaveAsync`, retry), `PlayerProgressData.cs` (the record), and `ProgressionUI.cs` (status and progress display). The local save is in `Assets/Scripts/SaveSystem`.

What the README explains. Project title, group members, description, controls, requirements and how to run the project, the folder structure, the database and data saved and retrieved, known limitations, and credits. *The group number and course are still placeholders in the README; references and tutorials used can be added if the instructor wants them in the README.*

How the repository is used. *[The group to describe: for example one commit per milestone with clear messages, which member commits what.]*

Part VI. Application Output Demonstration

Steps for the demonstration (live, or recorded if required). These steps were run in the Unity Editor on 11 October 2026; the level clear was triggered by a test command rather than by hand-playing the level.

#	Step	What to show
1	Open the game	Press Play on `00_MainMenu`. The "Connecting to Unity Cloud Save" screen appears, then the main menu. No login screen.
2	Play	Choose Play Adventure, pick a language and a level on the level map, move the character to an obstacle, answer the question.
3	Save data	Clear a level. The top-right message changes to "Saving to cloud..." and then "Saved to cloud".
4	View saved data	In the Unity Dashboard, open Cloud Save, then Player Data, find the Player ID and open `code_escape_progression`.
5	Retrieve and display	Stop and start the game again. The level map shows the cleared level and the unlocked next level, and the Cloud progress panel shows the saved values.

Observed in the Editor: the first run created a record and confirmed the save; a repeated level-clear event added no duplicate and sent no extra save; after a restart the record loaded and the level map showed Home completed, Neighborhood available and Warehouse locked.

Not yet tested by the group: a hand play-through of the questions, losing the network (error screen and Retry), and a standalone build. Video link, if required: *[link]*

Part VII. Systems Thinking Explanation

Component	Role in the Prototype
User or Player	A student who chooses levels, moves the character and answers programming questions.
Game Application	The Unity game client: menus, level map, gameplay, question system and the cloud service component.
Input	Keyboard and mouse input, the chosen language and level, and the answers submitted.
Process	Checking answers, calculating score, unlocking levels, building the progress record and comparing it with the last saved one.
Database or Storage	Unity Cloud Save (player data) for the cloud copy, and PlayerPrefs on the device for the local save.
Output	Question feedback, score, the level map, the save status message and the Cloud progress panel.
Network or API, if applicable	The internet connection and the Unity Gaming Services SDK, which call Unity Authentication and Unity Cloud Save. The group has no backend of its own.
Repository	GitHub repository holding the project files, README and history.
Development Tool	Unity Editor 6000.5.10f1 and a C# code editor; Unity Dashboard for the cloud project; GitHub Desktop for source control.

How this activity shows that a game application is a system of connected parts.

Code Escape is more than a game screen. A player gives input, the game logic turns it into results such as a score or an unlocked level, and those results become data. That data does not stay inside one script: an event announces it, the cloud service builds a record, the network carries the record to Unity Cloud Save, and Cloud Save keeps it under the player's identity. On the next start the same parts work backwards: authentication identifies the player, storage returns the record, the save system restores it, and the interface displays it. Each part depends on the others. If the network fails, the game cannot load and shows an error; if the data design is wrong, the level map shows the wrong levels; if the repository is missing a file, another developer cannot run it. The game is therefore a system with inputs, processes, storage, outputs and supporting tools that must fit together.

Part VIII. Group Reflection and Technical Learning

Draft based on the work done. The group should review and edit it so it reflects its own experience.

What the group achieved. The existing Code Escape game now saves progress to Unity Cloud Save and loads it when the game starts, without a login screen. The level map and a progress panel show the saved data, and the game blocks play with an error and Retry button when the cloud cannot be reached.

Technical learning.

- Cloud Save needs a signed-in identity. Anonymous sign-in gives one without a login screen, but it cannot recover progress on another device, so a login system is planned.
- Player data in Cloud Save is key-value JSON, not SQL tables, so the data design is a record structure rather than a table with relationships.
- Saving should be confirmed before the game says "saved". The game shows Unsaved, Saving, Saved or Failed, and keeps the local save as a backup.
- Repeated events can corrupt a score. Counting each solved challenge once, and removing duplicate level IDs, prevents double counting.
- Client-side data is not cheat-proof, because the game itself writes the score.
- Small configuration errors, such as a duplicated line in `manifest.json`, can stop the whole project from loading packages.

Limitations and next steps. The maps and gameplay are still in progress. The Login and Registration system and an administrator account that can change the coding questions are planned and not implemented. Network-failure behaviour and a standalone build still need testing.

Screenshots

The group must add the screenshots listed in Parts III, IV and VI here or in the parts above. Nothing has been fabricated.

- Game before saving; successful save message; Unity Dashboard showing the saved keys; game showing retrieved data; relevant code.
- Repository page on GitHub showing the files and the commit history.

References

- Unity Technologies. Unity Cloud Save documentation. docs.unity.com (Unity Gaming Services).
- Unity Technologies. Unity Authentication documentation (anonymous sign-in). docs.unity.com.
- Project documentation: `README.md` and `Documentation/CloudSave.md` in the repository.